

Special Topics in Discrete Mathematics: Advanced Topics in Cryptography

Theory Exercise 4

Panayi Andreas^{*}
MSc: A.L.MA.
Student ID: 711514240008

School of Electrical and Computer Engineering,
National Technical University of Athens
Athens, Greece

April 2026



National Technical
University of Athens

^{*}Writing was done using $\text{\LaTeX}$

Contents

Problem 1	1
Description	1
Solution	1
Problem 2	2
Description	2
Solution	2
Question 1	2
Question 2	3
Problem 3	5
Description	5
Solution	5
Question 1: Removing the timer.	5
Question 2: Using a 2Δ timer	5
Using a 4Δ timer.	6
Problem 4	7
Description	7
Solution	7

Problem 1

Description

Recall the longest chain PoS protocol, Ouroboros Praos, we studied in class. Ouroboros Praos is *available* but not *accountable*. We modify Ouroboros Praos to include a slashing mechanism: if the same key is used to create two blocks for the same height, we slash the key. Is the modified protocol t -accountably safe? If yes, prove it and find t . If not, is there another slashing mechanism that could make the protocol t -accountably safe?

Solution

The proposed same-height slashing rule. The modified Praos protocol is not t -accountably safe for any positive t . The proposed rule slashes a key only if it creates two distinct blocks at the same height. This rule may be sound, but it is not complete for accountability.

Indeed, a Common Prefix violation does not force any key to create two blocks at the same height. An adversarial key may build a private fork by producing blocks at successive heights, or the adversary may use different corrupted keys on the two competing forks. Thus two sufficiently deep incompatible chains may exist while no individual key violates the proposed rule.

Therefore, a safety violation may occur even though the proposed slashing rule does not identify any key. Hence the modified protocol is not 1-accountably safe, and therefore not t -accountably safe for any positive t . Its only trivial accountability threshold is $t = 0$.

General question of making Praos accountable. For the full Praos ledger, the answer is no in the general dynamically available model. Praos is available because it is a longest-chain protocol: progress does not require collecting votes from a fixed quorum of validators. If enough currently active stake keeps producing blocks, the chain can continue growing even when participation changes.

However, dynamic availability and accountable safety cannot both hold for the same ledger in the general dynamic-participation model. During a network partition, two disjoint groups of honest participants may each follow the protocol and extend incompatible histories. Since no participant necessarily violated the protocol, no sound slashing mechanism can identify a positive number of guilty participants. This is the availability-accountability dilemma [NTT22].

Consequently, one cannot make the full dynamically available Praos ledger t -accountably safe for any positive t merely by changing the block-level slashing rule.

What is possible is a two-layer construction (like what we saw in class). Praos can serve as the dynamically available ledger, while a separate quorum-based gadget periodically checkpoints a prefix. The full ledger remains available, whereas the checkpointed prefix can be final and accountable, but may lag behind or stop during partitions. This is the idea behind accountability gadgets and ebb-and-flow-style constructions [NTT22; NTT21].

Problem 2

Description

In this problem, we will consider the Simplex protocol run by n permissioned nodes, with a tunable quorum size q for notarization.

1. Suppose we want to maximize the resilience of the protocol, i.e., maximize the number of adversarial nodes f that can be tolerated such that the protocol is both safe and live. Derive the optimal quorum size q and show that the resulting optimal resilience is $f = n/3$.
2. Suppose you believe that an attack against safety is more likely than an attack against liveness (since a double-spend can provide significant rewards to the attacker). Hence, you want to tune Simplex to increase the resilience against safety attacks, even at the expense of decreasing the resilience against a liveness attack. Can this be done? If so, exhibit and plot the tradeoff between the two resiliences. If not, explain why not.

Solution

Question 1

Liveness condition. For liveness, the honest players must be able to notarize a block even if all Byzantine players refuse to vote. Since there are $n - f$ honest players, the quorum threshold must satisfy

$$q \leq n - f.$$

Safety condition. For safety, two conflicting blocks at the same height must not both become notarized.

First, we can derive the condition by counting votes. Two notarizations require $2q$ votes in total. Honest players can contribute at most $n - f$ of these votes, because an honest player does not vote for two conflicting non-dummy blocks. Hence, after using all possible honest votes, the number of votes that would still be needed is

$$2q - (n - f).$$

The Byzantine players can contribute at most $2f$ votes in total, because each Byzantine player may vote for both conflicting blocks. Therefore, to make two conflicting notarizations impossible, we need

$$2q - (n - f) > 2f.$$

Equivalently, $2q - n > f$, and so

$$q > \frac{n + f}{2}.$$

Second, we can derive the same condition by counting players. If Q_1 and Q_2 are two quorums of size q , then

$$|Q_1 \cap Q_2| \geq 2q - n.$$

This overlap must contain an honest player. Otherwise, the whole overlap could consist only of Byzantine players, who may vote for both conflicting blocks. Since there are at most f Byzantine players, we need

$$2q - n > f.$$

Thus, again,

$$q > \frac{n + f}{2}.$$

Combining the liveness and safety conditions , we need

$$\frac{n+f}{2} < q \leq n-f.$$

Such a quorum threshold exists only if

$$\frac{n+f}{2} < n-f.$$

Rearranging gives

$$n+f < 2n-2f,$$

and therefore

$$3f < n.$$

Thus the optimal Byzantine resilience is

$$f < \frac{n}{3}.$$

At the boundary $f \approx n/3$, the largest live quorum is

$$q = n - f \approx \frac{2n}{3}.$$

Therefore the optimal quorum threshold is $q = 2n/3$, up to integer rounding, and the protocol tolerates fewer than $n/3$ Byzantine players.

Question 2

Safety resilience. Let f_s denote the number of Byzantine players tolerated for safety. For a fixed quorum threshold q , two conflicting notarizations require $2q$ votes in total. The honest players can contribute at most $n - f_s$ of these votes, since an honest player does not vote for two conflicting non-dummy blocks. The remaining number of votes needed is therefore

$$2q - (n - f_s).$$

The Byzantine players can contribute at most $2f_s$ votes in total, since each Byzantine player may vote for both conflicting blocks. Thus, to rule out two conflicting notarizations, we need

$$2q - (n - f_s) > 2f_s.$$

Equivalently,

$$f_s < 2q - n.$$

Thus the safety resilience increases as q increases.

Liveness resilience. Let f_ℓ denote the number of Byzantine players tolerated for liveness. For liveness, the honest players must be able to notarize a block even if all Byzantine players refuse to vote. Since there are $n - f_\ell$ honest players, we need

$$q \leq n - f_\ell.$$

Equivalently,

$$f_\ell \leq n - q.$$

Thus the liveness resilience decreases as q increases.

Combining the two conditions. By tuning q , we can increase safety resilience at the expense of liveness resilience. The two conditions are

$$f_s < 2q - n \quad \text{and} \quad f_\ell \leq n - q.$$

To express the tradeoff directly, write $q = n - f_\ell$. Substituting into the safety condition gives

$$f_s < 2(n - f_\ell) - n,$$

and hence

$$f_s + 2f_\ell < n.$$

This tradeoff has a simple intuitive explanation. A larger quorum makes safety violations more difficult, since an adversary must gather more votes for each of two conflicting blocks. At the same time, it makes progress more fragile, because fewer unavailable or uncooperative nodes are sufficient to prevent any block from collecting enough votes. Conversely, lowering the quorum improves liveness but reduces the overlap between competing quorums and therefore weakens safety.

The asymmetry arises from the different ways Byzantine nodes affect the two properties. In a liveness attack, a Byzantine node can harm progress only by withholding its own vote. In a safety attack, however, the same Byzantine node may vote for both conflicting blocks and assist both notarizations simultaneously. This is why improving liveness consumes safety resilience at a faster rate than the reverse tradeoff.

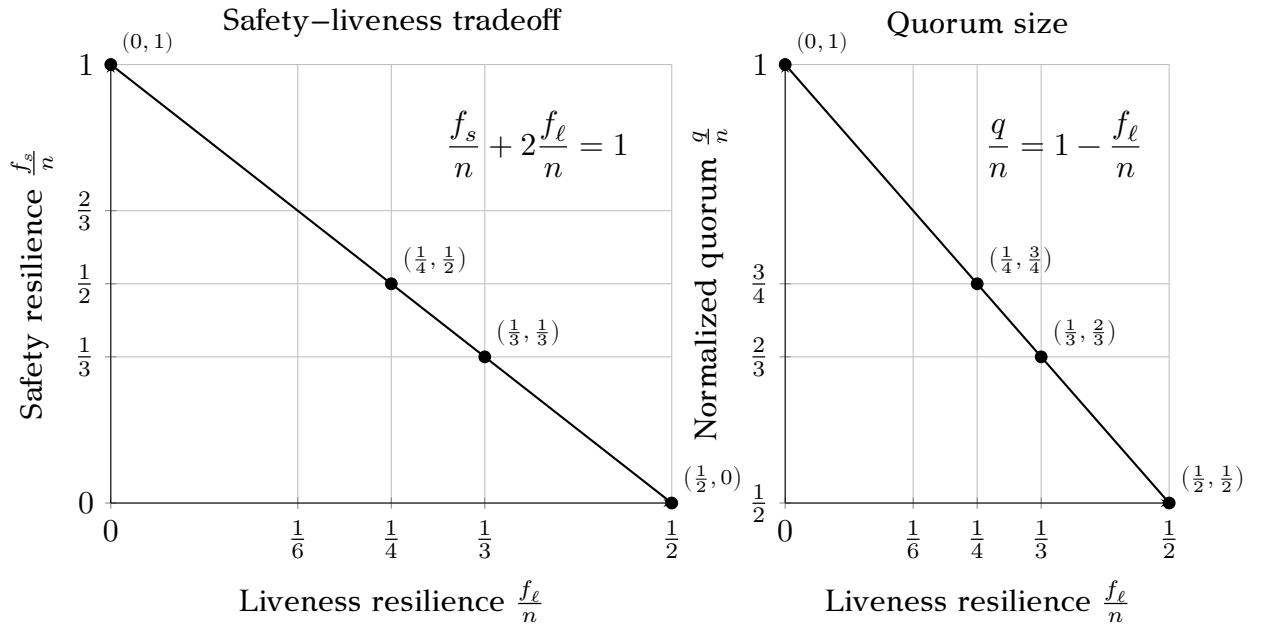


Figure 1: Left: the safety-liveness resilience boundary. Right: the quorum size corresponding to each liveness resilience.

Problem 3

Description

Consider the Simplex protocol we presented in class. Consider the following variations of the protocol and in each case explain if the security (safety and liveness) of the protocol is preserved. In each case, provide a proof that the protocol retains each virtue or a show a counterexample in which it fails.

1. We completely remove the timer of the protocol.
2. We change the timer from 3Δ to 2Δ .
3. We change the timer from 3Δ to 4Δ .

Solution

Question 1: Removing the timer.

Removing the timer preserves safety but destroys liveness. In the original protocol, a player leaves iteration h when it sees a notarized blockchain of length h . The timer is the backup mechanism: if the player is still in iteration h when the timer fires, it votes for the dummy block $\perp_h$. Thus, even if the leader is faulty, offline, or sends bad proposals, the dummy block can become notarized and allow honest players to move to iteration $h + 1$.

If the timer is removed, then this backup mechanism disappears. Hence, assuming that no ordinary block at height h becomes notarized, honest players have no way to leave iteration h . This can happen if the leader sends no proposal, sends only invalid proposals, sends proposals that do not extend a notarized chain of length $h - 1$, or splits honest votes among different proposals so that no proposal obtains a quorum. In all these cases, no height- h block becomes notarized. Since there is also no timer, honest players never vote for $\perp_h$, so no dummy block becomes notarized either. Therefore no notarized blockchain of length h is produced, and honest players may wait forever in iteration h .

Thus liveness fails. Safety, however, is preserved: removing the timer only removes the possibility of timeout votes for dummy blocks. It does not allow an honest player to vote for two conflicting non-dummy blocks at the same height. Therefore the usual quorum-intersection argument still prevents two conflicting non-dummy blocks at the same height from both being notarized.

Question 2: Using a 2Δ timer

A 2Δ timer is too short. The problem is not that the honest leader's proposal necessarily fails to arrive. Rather, the proposal may arrive and honest players may vote for it, but the votes may not have enough time to propagate and form a visible notarization before the timer fires.

Indeed, consider an honest player that enters iteration h first. Another honest player, possibly the leader of iteration h , may enter iteration h up to one message delay later. The leader then sends a valid proposal, which may take one more message delay to reach the first player. Thus the first player may receive the valid proposal only around time 2Δ . It can then vote for the proposal, but seeing enough votes to obtain a notarization may require one additional message delay.

Therefore, with a 2Δ timer, an honest player may vote for the leader's valid block and still have its timer fire before it sees the block notarized. It may then also vote for the dummy block $\perp_h$. Thus the timer is long enough for proposal delivery, but not long enough for vote delivery and

notarization.

This is precisely why the original protocol uses a 3Δ timer. The three delays correspond to: first, honest players becoming aligned in the iteration; second, the honest leader's proposal reaching the honest players; and third, the votes propagating so that the proposed block becomes notarized. With only 2Δ , the last step is missing, so timeout votes for the dummy block can coexist with votes for a valid honest proposal at the same height.

Using a 4Δ timer.

A 4Δ timer does not break the protocol. In particular, it does not hurt safety: making the timeout longer only delays dummy votes, and does not create a new way for two conflicting non-dummy blocks to be notarized. It also does not destroy liveness completely, since if the leader is faulty or silent, honest players will still eventually time out and vote for the dummy block $\perp_h$.

The problem is that 4Δ is an unnecessary delay. The honest-leader fast path only needs three delay windows: one for honest players to become aligned in the iteration, one for the leader's proposal to reach the honest players, and one for the votes to propagate and produce a notarization. Thus a 3Δ timer is already sufficient to avoid premature timeout during a successful honest-leader iteration. Using 4Δ simply makes faulty-leader iterations take one extra Δ before honest players vote for $\perp_h$. Therefore the protocol remains safe and live, but its liveness latency is worse than necessary.

Problem 4

Description

We modify the Simplex protocol to include a timestamp r in each non-dummy block. When an honest party creates a block, they put their clock's time as the timestamp r . In a valid chain, consecutive blocks have non-decreasing timestamps and no timestamp is in the future. We assume that honest parties have synchronized clocks and the network is synchronous. We are interested in how much the timestamp of the finalized tip of an honest party can deviate from their clock's time. Compute an upper bound v such that any honest party's tip will not deviate from their clock more than v except with negligible probability. Your bound does not need to be tight.

Solution

Quantity to be bounded. Let T be the current time according to an honest party's clock, and let r_{tip} be the timestamp of the most recent finalized non-dummy block in its chain. We seek a value v such that, except with negligible probability,

$$T - r_{\text{tip}} \leq v.$$

Since timestamps in a valid chain are non-decreasing, the timestamp becomes fresh whenever a block proposed by an honest leader is finalized.

What suffices to bound. It therefore suffices to bound the time between two successive finalized blocks proposed by honest leaders. Between such blocks, Byzantine leaders may propose blocks with old timestamps or may cause their iterations to end with dummy blocks, neither of which is guaranteed to refresh the timestamp.

Let

$$p = \frac{f}{n} < \frac{1}{3}$$

be the probability that the leader of an iteration is Byzantine. Assuming that leaders are chosen independently and uniformly, the probability that the next k iterations all have Byzantine leaders is

$$p^k < \left(\frac{1}{3}\right)^k.$$

Thus, choosing $k = \lambda$, where λ is the security parameter, gives

$$\Pr[\lambda \text{ consecutive Byzantine leaders}] < 3^{-\lambda},$$

which is negligible in λ .

Converting iterations into time. A Byzantine-led iteration lasts at most

$$3\Delta + \delta \leq 4\Delta,$$

because the honest parties time out after 3Δ and the dummy votes require at most one additional message delay to produce a notarized dummy block. Hence λ consecutive Byzantine-led iterations take at most

$$4\lambda\Delta.$$

Once an honest leader appears, its fresh block is finalized within at most

$$3\delta \leq 3\Delta.$$

Therefore, except with probability smaller than $3^{-\lambda}$,

$$T - r_{\text{tip}} \leq 4\lambda\Delta + 3\Delta.$$

Consequently, a valid non-tight freshness bound is

$$\boxed{v = (4\lambda + 3)\Delta}.$$

If the execution contains only polynomially many iterations in λ , a union bound over all possible starting iterations still gives a negligible failure probability. Thus, except with negligible probability, every honest party's finalized timestamp differs from its current clock by at most $O(\lambda\Delta)$.